

# TRACE

TRACKING, REPORTING, ANALYSIS & COMMUNITY EVIDENCE

## TRACE

Tracking, Reporting, Analysis & Community Evidence

Community vehicle tracking platform for neighborhood safety chapters. Reporters submit sightings from their phones. Operators triage, dispatch, and build intelligence from a desktop console. Three-vault database architecture ensures reporter identities stay separate from operational data.

[Deploy with Vercel](https://vercel.com/button)(https://vercel.com/new/clone?repository-url=https%3A%2F%2Fgithub.com%2Fduke-of-beans%2FTRACE&env=DATABASE\_URL&envDescription=Neon%20PostgreSQL%20connection%20string%20(pooler%20URL)&envLink=https%3A%2F%2Fneon.tech&project-name=trace&repository-name=TRACE)

You need a computer with a web browser. That's it. Everything runs in the cloud for free.

### Step 1: Create your database

Your database stores every sighting, vehicle, and dispatch record. It's private to you — no one else can access it. [Neon](https://neon.tech) provides free PostgreSQL databases that are encrypted at rest and in transit.

1. Go to [neon.tech](https://neon.tech) and sign up (Google account works, or create a fresh email/password)
2. Click **New Project**, name it anything (like `trace`), pick the region closest to your area
3. After it's created, you'll see a **Connection String** — it looks like gibberish, that's normal. Click the copy button next to it.

This connection string is like an address + key for your database. Keep it private.

### Step 2: Deploy TRACE

[Vercel](https://vercel.com) hosts your website for free — it's used by major companies and handles the infrastructure so you don't have to. Vercel needs a [GitHub](https://github.com) account (GitHub stores the code, Vercel turns it into a website). You won't need to learn GitHub — it's a one-time setup step.

1. Click the blue **Deploy with Vercel** button above
2. Sign in with GitHub (create a free account if you don't have one — just needs an email)
3. It asks for `DATABASE_URL` — paste the connection string you copied in Step 1
4. Click **Deploy** and wait about 2 minutes

When it finishes, you get a URL like [trace-abc123.vercel.app](https://trace-abc123.vercel.app). This is your TRACE. Bookmark it.

### Step 3: Set up the database tables

Your database exists but it's empty — like a filing cabinet with no folders. This step creates the structure TRACE needs to organize sightings, vehicles, dispatches, and accounts. You paste one file and click Run.

1. Go back to [Neon](https://console.neon.tech) and click on your project
2. Click **SQL Editor** in the left sidebar
3. Open [`setup.sql`](setup.sql) from this repository (click it, then click the copy button in the top-right)
4. Paste the entire contents into the Neon SQL Editor
5. Click **Run**

You'll see "CREATE TABLE" confirmations. Errors about things "already existing" are fine — it's safe to run multiple times.

### Step 4: Create your operator account

TRACE uses callsigns instead of email addresses to protect operator identity. Even if someone gained access to the database, they'd find code names, not real people.

1. Go to `your-app.vercel.app/operator/`
2. You'll see a **First-Time Setup** screen (only appears once — when no operators exist)
3. Enter your **chapter name** (e.g., "Westside Watch")
4. Choose a **callsign** — your operator code name (like "ALPHA" or "DISPATCH-1")
5. Choose an **access code** — at least 6 characters, keep it safe
6. Click **Create Operator & Start**

You're in. The onboarding walkthrough will show you around.

### Step 5: Invite your first reporter

Reporters don't create accounts with email or phone numbers — that would put them at risk if the system were ever compromised. Instead, you generate a one-time invite code and hand it to them directly.

1. In the operator console, click **Admin** in the sidebar
2. Click the **Reporters** tab
3. Type a callsign for the reporter (their code name, not their real name)

4. Click **\*\*Generate Invite Code\*\***

5. Send the code to your reporter via Signal, WhatsApp, or in person

The reporter opens your TRACE URL on their phone, enters the code, and they're in.

## Step 6: Install the app on phones

TRACE installs from the browser — no App Store or Google Play needed. This means no review process, faster updates, and no trail in anyone's app purchase history.

**iPhone (must use Safari):** Tap the Share button → Add to Home Screen → Add

**Android (use Chrome):** Tap the three-dot menu → Add to Home Screen → Add

The app icon appears on their home screen and runs full-screen like a native app.

> **Visual guide with mockups of every screen:** Once deployed, visit [your-app.vercel.app/guide.html](https://your-app.vercel.app/guide.html)

## For reporters

Open the app. Enter your invite code. You see a plate input and a map.

**Report mode.** Type a plate, describe the activity, drop the location, submit. Takes 15 seconds. The operator sees it immediately in their triage queue.

**Check Plate mode.** Type a plate to see if it is already in the database. If it matches, you can escalate to a full report with one tap.

**Map tab.** See active dispatch pins dropped by the operator. Tap a pin to respond. The operator sees your status (responding, on scene).

## For operators

The operator console is a desktop application at [/operator/](#).

**Triage.** Incoming sightings appear in a queue. Each one shows whether the plate matches a known vehicle (MATCH badge) or is new (NEW PLATE). Confirm and dispatch, dismiss with feedback to the reporter, or add the vehicle to tracking.

**Intel Map.** All sightings on a satellite map. Right-click to drop a dispatch pin. Click any sighting marker to see details. Time playback shows sighting patterns hour by hour. Corridor overlays trace vehicle movement paths.

**Vehicles and Actors.** Dossier pages for tracked vehicles and persons of interest. Photos, suspicion levels, sighting history, identifier records.

**Admin.** Configure vehicle types, suspicion ladders, dispatch event types, actor identifiers. Generate reporter invite codes.

Three-vault database

TRACE separates data into three PostgreSQL schemas. If someone gains access to the operational data (vehicles, sightings, actors), they cannot determine who submitted any report. Reporter real identities live in a separate schema with a separate encryption key.

| Schema     | Contains                                          | Access               |
|------------|---------------------------------------------------|----------------------|
| `ops`      | Vehicles, sightings, actors, dispatches, chapters | Full CRUD            |
| `ident`    | Reporter identities, auth tokens, sessions        | Auth service only    |
| `evidence` | Write-once evidence locker, hash chain            | INSERT + SELECT only |

A full dump of the [ops](#) schema reveals zero real identities. By architecture, not policy.

Stack

- **API:** Hono (TypeScript) on Vercel Serverless Functions
- **Database:** Neon PostgreSQL + Drizzle ORM
- **Reporter app:** Preact PWA (mobile-first, installable)
- **Operator console:** React SPA (desktop-first)
- **Maps:** Leaflet.js with Esri/CARTO/OSM tile layers
- **Auth:** Invite codes (no email required), TOTP for operators

Project structure

```
src/  
  api/  
    auth/  
      sightings/  
      vehicles/  
      actors/  
      dispatch/  
      geo/  
      admin/  
  db/  
    Hono route handlers  
    Invite-code and dev-login auth  
    Sighting intake + plate matching  
    Vehicle dossier CRUD  
    Actor profile management  
    Dispatch event lifecycle  
    Heatmap, corridors, co-occurrence, temporal  
    Chapter configuration + reporter invites
```

```

    schema/           Drizzle schema (three vaults)
    connection.ts     Three connection pools
    seed.ts           Demo data
    services/         Plate lookup, geospatial, notifications

    pwa/              Reporter PWA (Preact)
    src/pages/        Report, Map, History, Settings tabs
    src/components/   Onboarding, panic button, pin lock

    operator/         Operator console (React)
    src/pages/        Dashboard, Triage, Intel, Vehicles, Actors, Admin, Security
    src/components/   Map, time slider, UX primitives

    api/index.ts      Vercel serverless entry point
    shared/design/    Design tokens and theme

```

## Authentication model

Reporters authenticate with a one-time invite code generated by an operator. No email, no phone number, no password. The invite code is given in person or via an encrypted channel (Signal). Once used, the code is invalidated.

Operators authenticate with a callsign and access code. The first operator is created during initial setup (see Quick Deploy step 5).

## Production hardening checklist

Before giving anyone access to your TRACE instance:

- Set `TRACE\_DISABLE\_DEV\_LOGIN=true` in Vercel env vars
- Set `TRACE\_DISABLE\_TEST\_CODE=true` in Vercel env vars
- Change the default operator callsign from `OPERATOR` to something unique
- Verify the operator console rejects non-operator logins (it does by default)
- All access is logged to the audit table

## Who can see what

| Role     | Sees                                                                     | Does not see                                                        |
|----------|--------------------------------------------------------------------------|---------------------------------------------------------------------|
| Reporter | Their own submissions, active dispatch pins, plate check results         | Other reporters, operator actions, vehicle dossiers, actor profiles |
| Operator | All sightings (by callsign, not real name), vehicles, actors, dispatches | Reporter real identities, Vault B data                              |

Database admin

All three schemas

Nothing is hidden from the database admin

## Reporter safety

The reporter app includes a panic button (configurable in Settings) and a PIN lock. The operator can remotely suspend a reporter's device via the Security page, which sends a kill signal on the next status check.

## Prerequisites

- Node.js 22+
- PostgreSQL 15+ (local, Docker, or WSL)

## Setup

```
git clone <your-fork-url>
cd TRACE
npm install
cd pwa && npm install && cd ..
cd operator && npm install && cd ..
cp .env.example .env
# Edit .env with your local PostgreSQL credentials
```

## Database setup

```
# Create the database
psql -U postgres -c "CREATE DATABASE trace;"

# Run migrations
npx drizzle-kit migrate

# Seed demo data
npx tsx src/db/seed.ts
```

## Run

```
# API server (port 3100)
npm run dev

# Reporter PWA (port 5173)
cd pwa && npm run dev

# Operator console (port 5174)
cd operator && npm run dev
```

Demo credentials: operator callsign [OPERATOR](#), reporter invite code [TEST-CODE](#).

TRACE is a Progressive Web App. No app store needed.

#### **iPhone (Safari only)**

1. Open your TRACE URL in Safari
2. Tap the Share button (square with arrow)
3. Scroll down, tap "Add to Home Screen"
4. Tap "Add"

The app appears on the home screen with a standalone icon. It runs full-screen without Safari's address bar.

#### **Android (Chrome)**

1. Open your TRACE URL in Chrome
2. Tap the three-dot menu
3. Tap "Add to Home Screen" or "Install App"
4. Tap "Add"

#### **Desktop (Chrome/Edge)**

1. Open your TRACE URL
2. Click the install icon in the address bar (or three-dot menu > Install)

- [\[Visual Setup Guide\]\(guide.html\)](#) — interactive walkthrough with mockups (available at ``/guide.html`` on any deployed instance)
- [\[Chapter Setup Guide\]\(docs/CHAPTER\\_SETUP.md\)](#) — detailed deployment walkthrough
- [\[Dispatch System Design\]\(DISPATCH\\_DESIGN.md\)](#)
- [\[Voice Guide\]\(VOICE\\_GUIDE.md\)](#)
- [\[Design System\]\(DESIGN\\_SYSTEM.md\)](#)
- [\[Security Architecture\]\(docs/SECURITY\\_ARCHITECTURE.md\)](#)
- [\[Security Edge Cases\]\(docs/SECURITY\\_EDGE\\_CASES.md\)](#)
- [\[Requirements Synthesis\]\(docs/REQUIREMENTS\\_SYNTHESIS.md\)](#)

Source-available for community safety use. See repository for terms.

